

Minimum Spanning Trees & Kruskal's Algorithm

Algorithm Design & Analysis

August 18, 2026

What is a Minimum Spanning Tree (MST)?

Imagine you want to connect a group of computers using the cheapest possible cables.

A **Minimum Spanning Tree (MST)** is a network that has three rules:

- ▶ **Spanning:** It touches every single computer (node).
- ▶ **Tree:** There are NO loops or cycles. (If you have a loop, you can remove one cable to save money and still be connected!)
- ▶ **Minimum Cost:** The total price of all the cables used is the absolute lowest possible.

The Golden Rule: The Cut Property

How do we know which edges absolutely belong in the Minimum Spanning Tree? We use the **Cut Property**.

- ▶ **What is a Cut?** Imagine drawing a line that splits all the computers (nodes) into two separate groups (Group A and Group B).
- ▶ **Crossing Edges:** Look at all the cables (edges) that connect a computer in Group A directly to a computer in Group B.
- ▶ **The Rule:** The *absolute cheapest* cable that crosses this cut **must** be part of the Minimum Spanning Tree!

Why is this important?

This property is the mathematical proof behind Kruskal's Algorithm! It guarantees that our "Greedy" approach (always grabbing the cheapest available edge) is the correct choice.

Kruskal's Algorithm: The Simple Strategy

Kruskal's Algorithm builds the network using a "Greedy" approach. We always try to grab the cheapest cable available!

The 4 Steps:

1. **Sort** all edges from the lightest (cheapest) to the heaviest.
2. **Pick** the next cheapest edge.
3. **Check:** Does adding this edge create a loop?
 - ▶ If YES → Throw it away.
 - ▶ If NO → Add it to your network.
4. **Repeat** until all computers are connected.

The Cycle Problem

The big question: In Step 3, how do we quickly know if adding an edge creates a loop?

If we use standard graph algorithms (like DFS or BFS) to check for loops every single time, our program will be too slow.

We need to instantly answer: *“Are computer A and computer B already connected in our current network?”*

The Solution: Disjoint Set Union (DSU)

A clever data structure that keeps track of connected components.

DSU: The "Team Leader" Analogy

Think of DSU like assigning people to teams. Every team has one "Leader".

Three Simple Operations:

1. **MakeSet:** In the beginning, every node is alone. Everyone is the leader of their own 1-person team.
2. **Find:** Asks, *"Who is the leader of my team?"*
 - ▶ **Magic trick:** If Node A and Node B have the **same leader**, they are already in the same team! Adding a new cable between them would make a loop!
3. **Union:** Connects two teams together. We just make the leader of Team A follow the leader of Team B.

Merging Teams: Union by Rank vs. Size

When we merge two teams together, who becomes the new leader? If we do it randomly, our tree might become a long, slow chain.

- ▶ **Union by Size:** We track the *total number of nodes* in each team. We always attach the smaller team under the larger team.
- ▶ **Union by Rank:** We track the *height (depth)* of the tree. We always attach the shorter tree under the taller tree so the overall height doesn't increase.

Both strategies are extremely fast. We usually default to Union by Size because knowing the total size of a component is very useful for other graph problems!

C++ Implementation: DSU (By Size)

```
1 struct DSU {
2     vector<int> parent, sz;
3
4     DSU(int n) {
5         parent.resize(n);
6         sz.resize(n, 1); // Start with team size 1
7         for(int i = 0; i < n; i++) parent[i] = i;
8     }
9
10    int find(int i) {
11        if (parent[i] == i) return i;
12        return parent[i] = find(parent[i]); // Path compression
13    }
14
15    void unite(int i, int j) {
16        int root_i = find(i);
17        int root_j = find(j);
18        if (root_i != root_j) {
19            // Union by size: attach smaller to larger
20            if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
21            parent[root_j] = root_i;
22            sz[root_i] += sz[root_j]; // Update size of new root
23        }
24    }
```

Speeding it up: Path Compression

Notice line 12 in the previous code: `parent[i] = find(parent[i]);`

- ▶ **Without Compression:** Finding the leader could take $O(N)$ time for every check if the tree becomes a long chain. Kruskal's would be very slow.
- ▶ **With Compression:** When we find the leader, we update everyone on the path to point **directly** to the leader.
- ▶ **Result:** The tree becomes incredibly flat. Finding the leader now takes almost constant time, $O(1)$, on average!

Alternative C++ Implementation: Union by Rank

If you prefer to keep the trees as flat as possible strictly by height, here is how the unite function changes using a rank array (initially all 0):

```
1 void unite(int i, int j) {
2     int root_i = find(i);
3     int root_j = find(j);
4     if (root_i != root_j) {
5         // Attach the shorter tree under the taller tree
6         if (rank[root_i] < rank[root_j]) {
7             parent[root_i] = root_j;
8         } else if (rank[root_i] > rank[root_j]) {
9             parent[root_j] = root_i;
10        } else {
11            // If heights are equal, attach one to the other...
12            parent[root_j] = root_i;
13            // ...and increase the height of the new root by 1
14            rank[root_i]++;
15        }
16    }
17 }
```

C++ Implementation: Kruskal's Algorithm

```
1 struct Edge { int u, v, weight; };
2
3 // Helper function to tell C++ how to sort edges by weight
4 bool compareEdges(Edge a, Edge b) { return a.weight < b.weight; }
5
6 int kruskal(int V, vector<Edge>& edges) {
7     // 1. Sort edges from lightest to heaviest
8     sort(edges.begin(), edges.end(), compareEdges);
9
10    DSU dsu(V);
11    int total_cost = 0;
12
13    // 2. Iterate through sorted edges
14    for (Edge e : edges) {
15        // 3. If they have different leaders, no cycle is formed!
16        if (dsu.find(e.u) != dsu.find(e.v)) {
17            dsu.unite(e.u, e.v);    // Merge them
18            total_cost += e.weight; // Add cable cost to total
19        }
20    }
21    return total_cost;
22 }
```

Time Complexity

Why is this algorithm so good?

- ▶ **Sorting the Edges:** $O(E \log E)$
- ▶ **DSU Find/Union:** Because of "Path Compression", checking for loops takes almost constant time, $O(1)$.

Overall Speed

The speed of the algorithm depends entirely on the sorting step. Therefore, the total time complexity is $O(E \log E)$.

Concept Check:

Problem: Given a graph $G = (V, E)$ and an MST $T = (V, E')$. An edge e has its weight modified from $w(e)$ to $\hat{w}(e)$.

Update T to the new MST T' in **linear time** $O(|V| + |E|)$ for all four cases:

- (a) $e \notin E'$ and $\hat{w}(e) > w(e)$ (Non-tree edge weight *increased*)
- (b) $e \notin E'$ and $\hat{w}(e) < w(e)$ (Non-tree edge weight *decreased*)
- (c) $e \in E'$ and $\hat{w}(e) < w(e)$ (Tree edge weight *decreased*)
- (d) $e \in E'$ and $\hat{w}(e) > w(e)$ (Tree edge weight *increased*)

Solution: Cases (a) and (c) — No Structure Change

Case (a): $e \notin E'$ and $\hat{w}(e) > w(e)$ (Non-tree edge gets heavier)

Algorithm: Leave tree T unchanged ($T' = T$). Time: $O(1)$.

Proof: By the Cycle Property, e was already the heaviest edge on a cycle C in $T \cup \{e\}$. Making e heavier keeps it strictly heaviest on C , so $e \notin T'$.

Case (c): $e \in E'$ and $\hat{w}(e) < w(e)$ (Tree edge gets lighter)

Algorithm: Leave tree T unchanged ($T' = T$). Time: $O(1)$.

Proof: T was optimal. Decreasing $w(e)$ reduces T 's total cost by $\Delta = w(e) - \hat{w}(e)$. Any other tree using e drops by Δ , and trees without e don't drop. T remains the minimal tree.

Solution: Case (b) — Non-Tree Edge Gets Lighter

Case (b): $e = (u, v) \notin E'$ and $\hat{w}(e) < w(e)$

Algorithm ($O(|V|)$ Time):

1. Adding e to T forms a unique cycle C in $T \cup \{e\}$.
2. Find cycle C using BFS/DFS on T from u to v in $O(|V|)$ time.
3. Identify $e_{\max}$, the edge with maximum weight on C .
4. If $\hat{w}(e) < w(e_{\max})$, set $T' = (T \cup \{e\}) \setminus \{e_{\max}\}$. Else $T' = T$.

Proof of Correctness: By the Cycle Property, the maximum-weight edge on any simple cycle cannot belong to any MST. Removing $e_{\max}$ breaks cycle C and yields a valid spanning tree T' of minimal total weight.

Solution: Case (d) — Tree Edge Gets Heavier

Case (d): $e = (u, v) \in E'$ and $\hat{w}(e) > w(e)$

Algorithm ($O(|V| + |E|)$ **Time**):

1. Removing e splits T into two subtrees, defining a cut $(S, V \setminus S)$. Find S using BFS/DFS in $O(|V|)$ time.
2. Scan all edges E to find those crossing cut $(S, V \setminus S)$ in $O(|E|)$ time.
3. Find $e_{\min}$, the edge with minimum weight crossing $(S, V \setminus S)$ (considering updated $\hat{w}(e)$).
4. Set $T' = (T \setminus \{e\}) \cup \{e_{\min}\}$.

Proof of Correctness: Any valid spanning tree must contain an edge crossing cut $(S, V \setminus S)$ to maintain connectivity. By the Cut Property, the minimum-weight edge crossing any cut **must** belong to the MST. Thus, replacing e with $e_{\min}$ guarantees T' is an MST.